Звіт: злиття двох відсортованих послідовностей (варіант 9, *merge*)

Автор: Грабовський Богдан, К-27

1. Умови експерименту

- **Машина:** AMD Ryzen 5 7640HS, 6 фізичних ядер / 12 потоків (*hardware_concurrency* = 12), ОЗП 16 ГБ, Windows 64-bit.
- **Компілятор:** GCC 15.2.0 (MSYS2 MinGW-w64), C++20 (gnu++20), CMake + Ninja, Intel TBB; режим Release, рівні -O0, -O1, -O2, -O3.
- **Дані:** випадкові цілі $1…10^8$ (*std::mt19937*, фіксований seed), відсортовані, у текстових файлах (числа через пробіл). Пари довжин: 10 000+15 000; 100 000+150 000; $10^6+1{,}5\cdot10^6$; $5\cdot10^6+7{,}5\cdot10^6$. Усі рівні оптимізації використовували **одні й ті самі файли**.
- **Вимірювання:** *steady_clock*, 1 прогрівальний запуск + 15 повторів (5 для найбільшого набору), медіана, мс. Кожен результат перевірено порівнянням зі стандартним *std::merge* (у всіх випадках збіг). Предикати для *merge* не досліджувались: за умовою їх використання тут нерелевантне.

2. Власний алгоритм

Перша послідовність ділиться на K рівних частин; для кожної межі *a[i]* у другій послідовності за допомогою *lower_bound* знаходиться відповідна межа. K потоків (*std::thread*) зливають свої пари відрізків одразу у свої ділянки вихідного масиву, тому фінального послідовного зливання немає.

3. Результати при -O3 (мс)

N разом	без політики	seq	par	par_unseq	власний (найкр. K)	найкраще K	K / 12	K, де ріст зупиняється
25 000	0,0142	0,0118	0,0298	0,0289	0,1320	1	0,08	1
250 000	0,6007	0,5727	0,1033	0,0861	0,3334	4	0,33	4
2 500 000	5,9454	5,5672	0,6847	0,7790	1,1119	11	0,92	9
12 500 000	29,5648	28,3467	4,0823	4,4350	4,2153	11	0,92	10

Вплив оптимізації, N = 12 500 000 (мс; для власного алгоритму в дужках найкраще K):

Режим	-O0	-O1	-O2	-O3	-O0 → -O3
без політики	81,47	30,74	31,46	29,56	2,8×
seq	106,70	30,21	29,03	28,35	3,8×
par	19,74	4,29	4,15	4,08	4,8×
par_unseq	18,14	4,35	4,08	4,44	4,1×
власний (K)	9,80 (11)	4,46 (11)	4,41 (9)	4,22 (11)	2,3×

Повні таблиці «K → час → прискорення» для всіх розмірів: *results_O0.txt … results_O3.txt*.

4. Відповіді на запитання завдання

- **Найкраще K:** для найбільших даних K = 9–11 залежно від рівня оптимізації (при -O3: 11), тобто K / threads ≈ 0,75–0,92. Для малих даних K = 1–2, для 250 000 елементів K = 4–8.
- **Де швидкість перестає зростати:** при K ≈ 9–11 (≈ 0,75–0,9 від числа потоків). Прискорення відносно K=1 при -O3 виходить на ≈ 7,2× ($5\cdot10^6+7{,}5\cdot10^6$) і ≈ 5,5× ($10^6+1{,}5\cdot10^6$); після K = 12 покращення немає.
- **Вплив оптимізації:** -O0 → -O1 дає найбільше (2,8–3,8× для *merge*, 4–5× для *par*); -O1, -O2, -O3 практично однакові (різниця в межах шуму 5–10%).
- **Політики:** *seq* ≈ без політики на -O1…-O3 (≤ 8%); на -O0 *seq* повільніший на 31% (107 проти 81 мс). *par* і *par_unseq* на великих N у ≈ 7 разів швидші (на -O0 лише в 4–4,5×), на 25 000 елементах *par* навіть повільніший за звичайний *merge* (0,48×).

5. Висновки

- **Масштабування обмежене.** Прискорення ≈ 7× при 6 ядрах / 12 потоках. Це більше за число ядер, бо SMT ховає затримки пам'яті та гілкування, але менше за 12×: *merge* виконує одне порівняння й один запис на елемент і впирається в пропускну здатність пам'яті. На середніх розмірах (250 000–$2{,}5\cdot10^6$), де дані поміщаються в кеш, *par* дає 5–9×.
- **Малі дані.** Для 25 000 елементів *merge* триває ≈ 0,014 мс. Створення одного потоку коштує ≈ 0,1 мс (різниця K=1 і *merge* при -O3: 0,132 проти 0,014 мс), кожен наступний потік додає ≈ 0,033 мс (при K = 48 — 1,7 мс). Тому найкраще K = 1, а *par* не дає виграшу.
- **Чому K ≈ число потоків.** При K ≤ 12 усі порції виконуються одночасно, і більше K зменшує частку кожної. Далі час різко погіршується: при -O3 між K = 11 і 12 (4,22 → 6,02 мс), при -O2 між K = 10 і 11 (4,43 → 6,76 мс); точне K трохи «пливе» між запусками, бо головний потік і ОС теж потребують процесорного часу, а частина потоків мусить виконати дві порції послідовно. Біля K ≈ 20–22, де кожен

потік отримує по дві порції, час знову близький до оптимального (4,4–4,6 мс). Тому K слід брати трохи меншим за кількість потоків або кратним їй.

- **Власний алгоритм проти *par*.** На найбільших даних вони майже рівні (4,22 проти 4,08 мс при -O3, різниця ≈ 3%). На $2{,}5 \cdot 10^6$ *par* швидший (0,68 проти 1,11 мс), бо TBB перевикористовує пул потоків, а власний код створює K потоків при кожному виклику.
- ***par_unseq* не кращий за *par*:** *merge* має залежності за даними й гілки, тому векторизація нічого не дає.
- **Оптимізація.** Без оптимізації шаблонні ітератори й обгортки політик не інлайняться (*seq* на 31% повільніший за звичайний *merge*). Починаючи з -O1 код упирається в пам'ять, тому -O2/-O3 нічого не додають.
- **Похибка:** повторні запуски програми відрізняються на 5–15% (на найменших N — до 2×); окремі викиди (наприклад K = 4 при -O1: 12,5 мс замість ≈ 7,9) спричинені планувальником ОС. Наведено результати одного повного запуску (файли results_O*.txt).